

TravelMind: A Multi-Agent AI System with Deep Learning and LLM Reasoning for Personalized Travel Planning

Paladi Raghu* and K.Krishna Rani Samal[†]

Abstract—With the growing popularity of travel across the world, there is a sincere appetite for intelligent instruments which can assemble travel plans based on real-time information and enable the creation of personalized travel plans. Most current platforms still lean on rigid rule-based filters or simple models that struggle with the real complexity of planning: flights, accommodation, attractions, weather, local events and finances. In this paper, We present TravelMind, a multi-agent AI system where all the agents have distinct specialized domains and each relies on different deep learning backbones to construct comprehensive travel plans. The system integrates: (1) a SASRec Transformer paired with RL for flight recommendation and ranking; (2) Hotel-Related Knowledge Graph Neural Network Based Neural Collaborative Filtering; (3) a Transformer encoder for attractions having cross attention; and (4) a Bi-LSTM/GRU model to learn the weather-aware schedules; and (5) a BERT inspired classifier to keep track of events; and (6) intelligent budgeting via a DQN which has LSTM price prediction; (7) Route optimization using Graph Attention Networks with the help of RL; and (8) Large Language Model reasoning – applies reasoning around the entire problem threads it together. These agents are used in parallel with a central orchestrator that is responsible for taking them all into account and combining them appropriately. Our experiments demonstrate the superior performance of TravelMind in both improving recommendation quality, saving budgets, and planning a route, whilst maintaining response time under 15 seconds – and achieving this in a live system with Amadeus, Foursquare, OpenWeather and Ticketmaster.

Index Terms—Multi-Agent Systems, Deep Learning, Travel Recommendation, Transformer, Reinforcement Learning, Graph Neural Networks, Large Language Models, Neural Collaborative Filtering, Natural Language Processing, Intelligent Tourism

II. INTRODUCTION

Businesses in the tourism industry (valued at more than \$9.9 trillion USD each year by recent estimates [1]) are calling for “smarter digital tools.” Which isn’t surprising. When you’re planning a trip, there’s a lot to tune in to: Planning flights, booking a hotel, choosing interesting areas to explore, considering the weather forecast, searching for any local events to attend, keeping the finances straight, and figuring out how you can get from one destination to the next. All of these have their own data, quirks and live limitations.

The platforms most people use today—TripAdvisor, Google Travel, Booking.com, and the like—mostly rely on rule-based

filtering or collaborative filtering, and see each segment of a travel storyline as an independent entity from the other segments [2]. This will easily make three similar issues. They do not identify the cross dimensional relationships; e.g., if it’s going to be rainy, what kinds of attractions you are supposed to visit on that day. Second, most personalisation doesn’t extend beyond a few simple preferences that are customizable. Third, no one truly does generate a united, day-to-day schedule that takes on the job in a balanced approach.

Gayathri et al. [37] presented a machine learning framework to provide smarter and dynamic AI-based travel planning that generates itineraries in real-time. Their work demonstrates it is possible to plan travel using machine learning but was limited to a single model approach—no coordination from multiple agents, no deep learning specialization for each of the travel planning tasks.

Recent advancements in deep learning, reinforcement learning, and large language models [7] seem there is a viable chance to address such issues with bespoke domain-specific AI model. However, the truth is, there’s no one single architecture that performs well when it comes to all the components of travelling planning. That realization is what led us to go towards utilizing a multi-agent design, the foundations of which was set by [37] and developed into a fully-fledged multi-agent deep learning system.

In this paper, this intelligent multi-agent travel planning system called **TravelMind** is presented. Main contributions are:

- 1) **Multi-Agent Orchestration Architecture:** We suggest a framework of eight specialised agents communicating cooperatively under an orchestrating central agent that enables the second agent to run the remaining ones in parallel, but asynchronously.
- 2) **Diverse Deep Learning Integration:** Seven different neural network architectures (SASRec Transformer, NCF with KGNN, Transformer Cross-Attention, Bi-LSTM/GRU, BERT-style Encoder, DQN with Policy Gradient, and GAT with RL) are carefully selected for the different travel planning sub-tasks.
- 3) **LLM-Augmented Reasoning:** The use of a GPT-based large language model solves the problem of parsing preference for natural language, contextually identifying trips and intelligently mixing the deep learning results with the score.
- 4) **Real-Time Multi-Source Data Fusion:** The system connects to 5 external APIs (Amadeus, Foursquare, Open-

*Paladi Raghu is with the Department of Artificial Intelligence and Machine Learning, SCOPE, Vellore Institute of Technology (VIT), Vellore 632014, Tamil Nadu, India (e-mail: paladi.raghu2025@vitstudent.ac.in).

[†]K.Krishna Rani Samal is with the Department of Computational Intelligence, SCOPE, Vellore Institute of Technology (VIT), Vellore 632014, Tamil Nadu, India (e-mail: Krishnarani.samal@vit.ac.in).

Weather, Ticketmaster, and Open-AI) and gracefully fails over and has fall-backs built-in.

- 5) **End-to-End System Implementation:** We provide a production-ready Stack comprised of FastAPI back-end, MongoDB, ChromaDB database and a reactive WebUI that is capable of conversational AI interactions.

Everything discussed in the rest of the paper is organized as follows: Section III presents the related work; Section IV presents our system architecture; Section V presents one by one the description of each agent from the system and the corresponding Deep Learning model; Section VI presents implementation details; Section VII presents experimental results; Section VIII presents a discussion over our results and Section IX presents conclusions of this paper.

III. RELATED WORK

A. Travel Recommendation Systems

Over the years, travel recommendation systems have underwent a lot of development from the first collaborative filtering and content-based systems [2], [3] in the past number of years. The earlier work (nicely reviewed by Gavalas et al. [4]) was based on preference and demography filters and matched a traveler with a destination. Recently, researchers have been focusing on deep learning in order to go further with personalization. For instance, Zheng et al. [5] proposed attention-aware models for PWD recommendation and Chen et al. [6] introduced the idea of ‘context-aware sequential recommendation’ to the realm via RNNs. In the personal tour recommendation problem where the goal is to devise personalized tour plans, Lim et al. [8] came up with a novel solution of modeling how popular a particular place is and also how interested a particular user is, which they demonstrated to be effective in drawing up personalized tour plans based on data.

But, most of the systems focus on a single part of the travel process, for example, they aim at predicting a user’s most likely next destination (POI recommendation) or the next hotel to stay at (hotel ranking), instead of dealing with the complex, tangled, all-rounded process of a user planning a travel journey.

B. Multi-Agent Systems in Tourism

Multi-agent systems (MAS) have appeared in numerous areas including robotics, self-driving cars, shopping and other online markets [9]. For the tourism area, Camacho et al. [10] were among the first ones to propose agent based research in travel planning and Schiaffino et al. [11] have presented agent architectures for e-tourism applications. In recent years, Zhang et al. [12] demonstrated that with the aid of the LLM, the agents can split complicated tasks into easy-to-process steps quite well. Park et al. [13] showed that generative agents can exhibit realistic human behaviour which opens up interesting possibilities in terms of agents that allow for high level personalisation.

We offer each agent its specific deep learning model, as opposed to the various agents sharing the same LLM backbone. This will enable each agent to work for their specific role.

C. Sequential Recommendation Models

Self-Attentive Sequential Recommendation, or SASRec [14], introduced a self-attention module into the recommendation system, and proved to be superior to the previous methods based on RNNs. Ever since then, the Transformer architecture [15] has been rapidly adopted as a standard paradigm by the main systems in the recommender systems community such as BERT4Rec [16] and S³-Rec [17]. Pointing to this out, it’s important to mention the work of Hidasi et al. [18] as the pioneers in creating session-based recommendation based on recurrent networks, and how the attention-based methods on which Hidasi et al.’s work founded later followed.

D. Graph Neural Networks for Routing

A Graph Attention Network (GAT with focus on the GAT) [19] has been shown to capture structural relationships of data, which is useful. They have been used to solve routing and scheduling problems: Joshi et al. [20] introduced GNNs to answer Travelling Salesman Problem (TSP) and Kool et al. [21] presented Attention-based models to learn routing problems (RP).

E. Neural Collaborative Filtering

Together, He et al. [22] merged together matrix factorization and deep neural networks into a single entity called Neural Collaborative Filtering (NCF) for recommendation problems. This can be improved by extending it with knowledge graphs [23] as graph-structured embeddings can embed rich relationships between item attributes.

F. Reinforcement Learning for Budget Optimization

In the area of resource allocation, interesting uses of deep Q-Networks [24] and policy gradient methods [25] have been employed. In the travel field, RL is applied to dynamic pricing [26] and Itinerary planning [27] problems particularly. Our budget module uses the policy gradient part of the REINFORCE algorithm introduced by Williams [28].

G. Large Language Models in Planning

Large Language Models like GPT [29] have shown a remarkable knack for reasoning, and that extends to planning and decision-making contexts. Inspired by Wei et al. [30] who proposed chain of thought prompting to enhance complex reasoning, we apply this method to create structured trip planning guides from LLMs. To help complete tasks in environments involving more than one planning step, Yao et al. [31] introduced the ReAct framework by integrating reasoning and acting within LLM agents, leading to a remarkable improvement in their task success rates.

H. Taxonomy and Comparative Analysis

To clearly position TravelMind in the landscape of intelligent tourism research, Table I provides a comparative taxonomy comparing TravelMind against key prior literature across critical architectural capabilities.

TABLE I: Taxonomy and Architectural Feature Comparison of TravelMind Against Existing Travel Recommendation and Planning Systems.

System / Study	Multi-Agent	Deep Learning Backbones	LLM Reasoning	Async Orchestration	Graph Routing	RL Budgeting	Real-Time APIs	End-to-End Itinerary
Borràs et al. [2]	No	Rule / Heuristic	No	No	No	No	No	Single POI
Lim et al. [8]	No	Popularity / Heuristic	No	No	Integer Prog.	No	No	Tour Route Only
Zheng et al. [5]	No	GCN	No	No	No	No	Static	Place Rec Only
Chen et al. [6]	No	RNN + Memory Net	No	No	No	No	Static	Sequential Rec
Camacho et al. [10]	Yes	None (Rule-Based)	No	Sequential	Basic TSP	No	Web Scraping	Multi-Item List
Zhang et al. [12]	Yes	LLM Only	Yes	Partial	No	No	No	Symbolic Plan
Park et al. [13]	Yes	LLM Only	Yes	Sequential	No	No	No	Simulation Only
Gayathri et al. [37]	No	Single ML Model	No	Sequential	Heuristic	Fixed Rules	Static	Basic Itinerary
Kool et al. [21]	No	Attention Net	No	N/A	Pointer RL	No	Synthetic	Routing Only
Wang et al. [23]	No	KGAT	No	N/A	No	No	Static	Recommendation Only
Yao et al. [31]	Single Agent	LLM Only	Yes	No	No	No	Search API	Textual Plan
TravelMind (Ours)	Yes (8 Agents)	7 Specialized DL Models	Yes (GPT-4)	Yes (async)	GAT + RL	DQN + Policy Grad.	Yes (5 APIs)	Full Integrated Plan

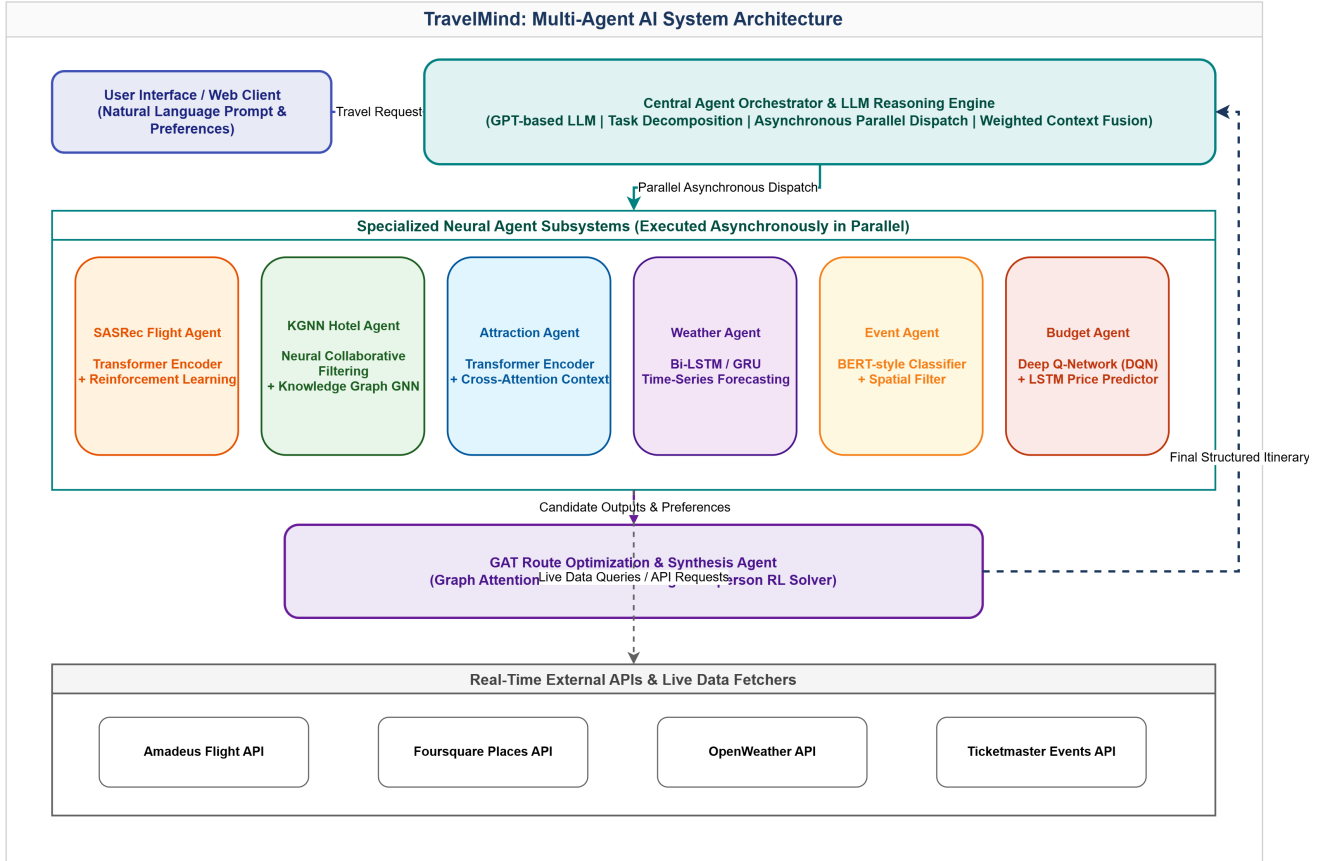


Fig. 1: Comprehensive High-Level System Architecture of TravelMind showing the 3-tier layout, central asynchronous orchestrator, 8 specialized deep learning/LLM agents, and persistent data/API integration layers.

IV. SYSTEM ARCHITECTURE

A. Overview

From a high level, TravelMind is split into three components—a presentation tier (web frontend), an application tier, which includes the FastAPI backend and the travel agent orchestrator, and finally a data tier (MongoDB, ChromaDB and the different external APIs). The overall layout is displayed in Fig. 1.

B. Agent Orchestrator

The Agent Orchestrator is a centre brain of the operation. It has a six-phase pipeline to synchronously control all of them:

- 1) **Preference Extraction:** When a user types in what they want, the system runs their input through both a rule-based

NLP extractor and an LLM-based structured parser to pull out interests, budget range, travel dates, and any special requirements.

- 2) **Parallel Data Collection:** Five agents—Flight, Hotel, Place, Weather, and Event—fire off simultaneously using Python’s `asyncio.gather()`, hitting external APIs and running ML model scoring at the same time.
- 3) **Distance Optimization:** Once places are collected, the Distance Agent kicks in with its GAT-based route optimizer to figure out a sensible visiting order.
- 4) **Data Assembly:** All the results get pulled together into a single, unified travel context object.
- 5) **Parallel Generation:** Itinerary generation, budget optimization, and travel summary creation all happen concurrently.

Algorithm 1 Asynchronous Multi-Agent Orchestration & Data Fusion

Require: User natural language prompt U_p , explicit filter vector $\mathbf{v}_{\text{user}}$, API keys $\mathcal{K}$

Ensure: Unified Personalized Itinerary Object $\mathcal{I}_{\text{final}}$

- 1: {**Phase 1: Natural Language Preference Extraction**}
 - 2: $\mathcal{P}_{\text{rule}} \leftarrow \text{ExtractRegexPatterns}(U_p)$
 - 3: $\mathcal{P}_{\text{llm}} \leftarrow \text{AsyncGPTParse}(U_p, \mathbf{v}_{\text{user}})$
 - 4: $\mathcal{P}_{\text{user}} \leftarrow \text{MergePreferences}(\mathcal{P}_{\text{rule}}, \mathcal{P}_{\text{llm}})$
 - 5: {**Phase 2: Asynchronous Agent Execution**}
 - 6: $\mathcal{T} \leftarrow [\text{AsyncFlightAgent}(\mathcal{P}_{\text{user}}), \text{AsyncHotelAgent}(\mathcal{P}_{\text{user}}), \text{AsyncPlaceAgent}(\mathcal{P}_{\text{user}}), \text{AsyncWeatherAgent}(\mathcal{P}_{\text{user}}), \text{AsyncEventAgent}(\mathcal{P}_{\text{user}})]$
 - 7: $\mathcal{R}_{\text{raw}} \leftarrow \text{await } \text{asyncio.gather}(*\mathcal{T}, \text{return_exceptions} = \text{True})$
 - 8: $(\mathbf{F}_{\text{flt}}, \mathbf{H}_{\text{htl}}, \mathbf{P}_{\text{plc}}, \mathbf{W}_{\text{wtr}}, \mathbf{E}_{\text{evt}}) \leftarrow \text{HandleFailover}(\mathcal{R}_{\text{raw}})$
 - 9: {**Phase 3: Spatial Distance Optimization**}
 - 10: $\mathbf{R}_{\text{route}} \leftarrow \text{await } \text{AsyncDistanceAgent}(\mathbf{P}_{\text{place}})$
 - 11: {**Phase 4 & 5: Context Assembly & Parallel Generation**}
 - 12: $\mathcal{C}_{\text{ctx}} \leftarrow \text{AssembleContext}(\mathbf{F}_{\text{flight}}, \mathbf{H}_{\text{hotel}}, \mathbf{R}_{\text{route}}, \mathbf{W}_{\text{weather}}, \mathbf{E}_{\text{event}})$
 - 13: $\mathcal{T}_{\text{gen}} \leftarrow [\text{AsyncBudgetAgent}(\mathcal{C}_{\text{ctx}}), \text{AsyncItineraryAgent}(\mathcal{C}_{\text{ctx}})]$
 - 14: $(\mathbf{B}_{\text{budget}}, \mathcal{I}_{\text{llm}}) \leftarrow \text{await } \text{asyncio.gather}(*\mathcal{T}_{\text{gen}})$
 - 15: {**Phase 6: Response Compilation**}
 - 16: $\mathcal{I}_{\text{final}} \leftarrow \text{CompileFinalResponse}(\mathcal{C}_{\text{ctx}}, \mathbf{B}_{\text{budget}}, \mathcal{I}_{\text{llm}})$
 - 17: **return** $\mathcal{I}_{\text{final}}$
-

6) **Response Compilation:** Finally, the system puts together the complete response—recommendations, the itinerary, budget breakdown, AI-generated insights, and metadata.

C. Asynchronous Execution Model

The orchestrator is written using Python’s `asyncio` framework, making sure nothing needlessly stops. All agents execute as separate asynchronously running coroutines so that parallel calls to the API and parallel inference of models are possible. `return_exceptions=True` is an intentional option: it does say that if one agent fails to make it up to standard, the rest of the system hums on unaffected.

Algorithm 1 formally outlines the asynchronous multi-agent execution sequence.

V. SPECIALIZED AGENT DESIGN AND DEEP LEARNING MODELS

A. Flight Recommendation Agent (SASRec + RL)

Finding the right flight is handled by a two-step process: first, a sequential recommendation model encodes flight features, and then an RL optimizer refines the rankings based on budget fit.

1) *SASRec Transformer Encoder:* We build on the SASRec architecture [14] to encode flight features. Given a batch of n flights, each described by a feature vector $\mathbf{f}_i \in \mathbb{R}^8$, the encoder works as follows:

$$\mathbf{H} = \text{TransformerEncoder}(\mathbf{W}_p \mathbf{F} + \mathbf{P}) \quad (1)$$

Here, $\mathbf{F} \in \mathbb{R}^{n \times 8}$ is the matrix of flight features, $\mathbf{W}_p \in \mathbb{R}^{8 \times 64}$ projects those features into a 64-dimensional space, and $\mathbf{P} \in \mathbb{R}^{n \times 64}$ provides learnable positional embeddings. We use 4 attention heads across 2 layers, with GELU as the activation.

Established scoring head generates suggestion scores for every flight:

$$s_i = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h}_i + \mathbf{b}_1) + \mathbf{b}_2) \quad (2)$$

The SASRec encoder optimization minimizes binary cross-entropy over positive flight selections:

$$\mathcal{L}_{\text{SASRec}} = - \sum_{i=1}^n [y_i \log s_i^{\text{SASRec}} + (1 - y_i) \log(1 - s_i^{\text{SASRec}})] \quad (3)$$

These eight flight features are: the average price in the normalized price, departure hour, number of stops, flight length, the rating for the airline, booking class, price-to-average, and directness score.

2) *RL Q-Network Price Optimizer:* On top of the SASRec scores, a Deep Q-Network algorithm also computes a cost value for each flight based on user’s budget preferences:

$$Q(s, a) = \text{MLP}_{32 \rightarrow 32 \rightarrow 1}(\mathbf{s}) \quad (4)$$

The state vector $\mathbf{s} \in \mathbb{R}^6$ packs in normalized price, budget alignment, a penalty for stops, time preference, duration efficiency, and a carrier score.

We combine the two models with a weighted sum:

$$\text{score}_i = \alpha \cdot s_i^{\text{SASRec}} + (1 - \alpha) \cdot Q_i^{\text{RL}}, \quad \alpha = 0.6 \quad (5)$$

B. Hotel Recommendation Agent (NCF + KGNN)

1) *Neural Collaborative Filtering:* For hotels, we use the NCF framework [22], which runs two pathways in parallel:

Generalized Matrix Factorization (GMF):

$$\mathbf{p}^{\text{GMF}} = \mathbf{W}_u^{\text{GMF}} \mathbf{x}_u \odot \mathbf{W}_h^{\text{GMF}} \mathbf{x}_h \quad (6)$$

The user preference vector $\mathbf{x}_u \in \mathbb{R}^8$ captures things like budget encoding and interest categories. The hotel feature vector $\mathbf{x}_h \in \mathbb{R}^{10}$ includes price, rating, star level, distance from the city center, number of amenities, review score, location quality, whether it’s part of a chain, price-range fit, and category match.

MLP Tower:

$$\mathbf{p}^{\text{MLP}} = \text{MLP}_{18 \rightarrow 64 \rightarrow 32 \rightarrow 16}([\mathbf{x}_u; \mathbf{x}_h]) \quad (7)$$

2) *Knowledge Graph Neural Network:* Additionally, we provide a Knowledge Graph embedding layer for the relationship between different features of the hotel:

$$\mathbf{e}_{\text{kg}} = \text{LayerNorm} \left(\text{ReLU} \left(\mathbf{W}_{\text{kg}} \cdot \frac{1}{k} \sum_{j=1}^k \mathbf{E}_{\text{amenity}}[a_j] \right) \right) \quad (8)$$

The amenity embedding matrix $\mathbf{E}_{\text{amenity}} \in \mathbb{R}^{12 \times 16}$ covers 12 types of amenities—WiFi, pool, spa, gym, restaurant, parking, bar, laundry, business center, concierge, room service, and airport shuttle.

Putting it all together, the final score is:

$$\hat{y} = \sigma(\mathbf{W}_{\text{out}}[\mathbf{p}^{\text{GMF}}; \mathbf{p}^{\text{MLP}}; \mathbf{e}_{\text{kg}}]) \quad (9)$$

The multi-task objective combines collaborative filtering prediction loss with knowledge graph entity regularization:

$$\mathcal{L}_{\text{Hotel}} = \mathcal{L}_{\text{NCF}}(\hat{y}, y) + \lambda \|\mathbf{W}_{\text{kg}}\|_2^2 \quad (10)$$

C. Place Recommendation Agent (Transformer + Cross-Attention)

1) *Sequential Place Encoder*: A Transformer encoder handles the place features:

$$\mathbf{Z} = \text{LayerNorm}(\text{TransformerEncoder}(\mathbf{W}_p \mathbf{X}_{\text{place}})) \quad (11)$$

Each place is described by $\mathbf{X}_{\text{place}} \in \mathbb{R}^{n \times 8}$, capturing rating, category, a base score, whether tips are available, whether there's a description, distance, a popularity proxy, and time suitability.

2) *Interest-Place Cross-Attention*: To figure out how relevant each place is to the user's particular interests, we use cross-attention:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (12)$$

The queries $\mathbf{Q}$ come from the user's interest vector $\mathbf{x}_{\text{int}} \in \mathbb{R}^{12}$ (covering 12 interest categories), while the keys $\mathbf{K}$ and values $\mathbf{V}$ come from the encoded place embeddings $\mathbf{Z}$.

The final score blends ML and LLM outputs:

$$\text{score}_i = 0.6 \cdot s_i^{\text{ML}} + 0.4 \cdot s_i^{\text{LLM}} \quad (13)$$

D. Weather Forecasting Agent (Bi-LSTM + GRU)

Weather is inherently temporal, so the Weather Agent adopts the sequence-to-sequence structure to learn about temporal weather characteristics.

1) *Bidirectional LSTM Encoder*:

$$\vec{\mathbf{h}}_t, \overleftarrow{\mathbf{h}}_t = \text{Bi-LSTM}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}, \overleftarrow{\mathbf{h}}_{t+1}) \quad (14)$$

The encoder has 2 layers with a hidden dimension of 32. It processes weather feature vectors $\mathbf{x}_t \in \mathbb{R}^6$, which consist of a normalized temperature value and a 5-category one-hot encoding for weather conditions.

2) *GRU Decoder*: The GRU decoder works on the representations encoded by the encoder and refine them:

$$\hat{\mathbf{y}}_t = \sigma(\mathbf{W}_o \cdot \text{GRU}(\text{LayerNorm}(\vec{\mathbf{h}}_t \| \overleftarrow{\mathbf{h}}_t))) \quad (15)$$

The sequence loss minimizes Mean Squared Error over historical weather suitability labels:

$$\mathcal{L}_{\text{Weather}} = \frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{y}}_t - \mathbf{y}_t^*\|^2 \quad (16)$$

3) *Context-Aware Activity Model*: On top of the weather analysis, a self-attention-based activity scorer maps forecasted conditions onto 8 activity suitability scores. These cover outdoor sightseeing, museums and indoor activities, water-based activities, hiking and nature, shopping, dining, photography, and cultural events.

E. Event Awareness Agent (BERT-style + KG)

1) *Event Text Encoder*: A lightweight, BERT-inspired Transformer encoder processes event descriptions. It uses hash-based tokenization, adds positional embeddings, and runs through a 2-layer Transformer:

$$\mathbf{e}_{\text{event}} = \text{Classifier}(\text{TransformerEncoder}(\mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}})[0]) \quad (17)$$

2) *Interest-Event Matcher*: A bilinear attention mechanism computes how relevant an event is given the user's interests. It takes interest features and event type embeddings and produces personalized relevance scores:

$$s^{\text{match}} = \mathbf{x}_{\text{int}}^T \mathbf{W}_{\text{bilinear}} \mathbf{e}_{\text{event}} \quad (18)$$

3) *Event Knowledge Graph*: In addition, we construct a knowledge graph based on structural embedding to model the relation between 10 kinds of events and 8 kinds of interests, and use a relation-aware MLP to score them.

The event score is a function of three signals weighted to give the final event score:

$$\text{score} = 0.3 \cdot s^{\text{NLP}} + 0.4 \cdot s^{\text{match}} + 0.3 \cdot s^{\text{KG}} \quad (19)$$

F. Budget Optimization Agent (DQN + LSTM + Policy Gradient)

1) *Deep Q-Network for Allocation*: The Budget Agent treats budget allocation as a sequential decision-making problem. The DQN is set up as:

$$Q(\mathbf{s}, a) = \text{MLP}_{12 \rightarrow 64 \rightarrow 64 \rightarrow 7}(\mathbf{s}) \quad (20)$$

The state $\mathbf{s} \in \mathbb{R}^{12}$ packs in trip duration, number of travelers, budget level encoding, average prices for flights, hotels, and places, weather conditions, event density, interest diversity, and seasonal factors. The 7 output actions correspond to budget categories: flights, accommodation, food, activities, local transportation, events, and miscellaneous expenses.

2) *LSTM Price Predictor*: A 2-layer LSTM (hidden dimension 32) handles price trend prediction:

$$\hat{p}_{t+1} = \sigma(\mathbf{W}_p \cdot \text{LSTM}(\mathbf{x}_{1:t})) \quad (21)$$

3) *Policy Gradient Multi-Objective Optimizer*: A policy gradient network learns to output an optimal budget split:

$$\pi(\mathbf{s}) = \text{Softmax}(\text{MLP}_{12 \rightarrow 32 \rightarrow 32 \rightarrow 7}(\mathbf{s})) \quad (22)$$

The policy parameters θ are updated via REINFORCE policy gradient objective maximizing budget utilization efficiency R :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|\mathbf{s}) Q(\mathbf{s}, a)] \quad (23)$$

The final allocation will in practice be a combination of the DQN's Q-values and the distribution of the policy gradient, weighted by the predictions of LSTM.

Algorithm 2 GAT-Pointer RL Route Optimization

Require: Selected POI Set $\mathcal{P} = \{p_1, p_2, \dots, p_M\}$, Coordinates (lat_m, lon_m)

Ensure: Sequentially Optimized Route $\pi^* = (p_{\pi_1}, p_{\pi_2}, \dots, p_{\pi_M})$

- 1: Construct spatial graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with edge weights $d_{ij} = \text{Haversine}(p_i, p_j)$
- 2: Initialize node embeddings $\mathbf{h}_i^{(0)} \leftarrow \text{Concat}(lat_i, lon_i, score_i, duration_i)$
- 3: **for** layer $l = 0, 1$ **do**
- 4: **for** each node $i \in \mathcal{V}$ **do**
- 5: Compute attention weights α_{ij} via Eq. (22)
- 6: Update node representation $\mathbf{h}_i^{(l+1)}$ via Eq. (21)
- 7: **end for**
- 8: **end for**
- 9: $\mathcal{U} \leftarrow \{p_1, \dots, p_M\}$, $curr \leftarrow \text{SelectStartNode}(\mathcal{P})$, $\pi^* \leftarrow [curr]$, $\mathcal{U} \leftarrow \mathcal{U} \setminus \{curr\}$
- 10: **while** $\mathcal{U} \neq \emptyset$ **do**
- 11: **for** each candidate $k \in \mathcal{U}$ **do**
- 12: Compute transition probability $p(k|curr)$ via Eq. (23)
- 13: **end for**
- 14: $next_node \leftarrow \arg \max_{k \in \mathcal{U}} p(k|curr)$
- 15: Append $next_node$ to π^* , update $curr \leftarrow next_node$, $\mathcal{U} \leftarrow \mathcal{U} \setminus \{next_node\}$
- 16: **end while**
- 17: **return** Optimized route sequence π^*

G. Distance Optimization Agent (GAT + RL)

1) *Graph Attention Network:* The nodes of the local graph represent places, whose weights are computed based on the distances. For the node features there are two GAT processing stages:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_j^{(l)} \right) \quad (24)$$

The attention coefficients themselves are computed as:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W} \mathbf{h}_i \| \mathbf{W} \mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W} \mathbf{h}_i \| \mathbf{W} \mathbf{h}_k]))} \quad (25)$$

2) *RL Route Policy:* Taking inspiration from pointer networks, the RL policy picks the next place to visit:

$$p(j|i) = \frac{\exp(\mathbf{W}_{\text{ptr}}[\mathbf{h}_i; \mathbf{h}_j])}{\sum_{k \in \text{unvisited}} \exp(\mathbf{W}_{\text{ptr}}[\mathbf{h}_i; \mathbf{h}_k])} \quad (26)$$

This essentially tackles the TSP using learned spatial-temporal representations. For geographic accuracy, we compute actual distances using the Haversine formula. Algorithm 2 details the GAT route optimization procedure.

H. Itinerary Generation Agent (LLM Reasoning)

The Itinerary Agent gives everything to OpenAI's GPT to come up with the daily travel plan. It displays all the relevant data collected to you (weather, cost, places, events, etc.) and then, it asks a model to populate a detailed prompt and return

TABLE II: Technology Stack

Component	Technology
Backend Framework	FastAPI (Python 3.10+)
ML Framework	PyTorch
Database	MongoDB (Motor async driver)
Vector Database	ChromaDB
LLM Integration	OpenAI GPT API
Flight/Hotel API	Amadeus API
Places API	Foursquare Places API
Weather API	OpenWeather API
Events API	Ticketmaster API
Frontend	HTML5, CSS3, JavaScript

back a structured response in JSON with daily activities, meal recommendations, timing, cost estimates, and travel tips.

A deterministic fallback was also implemented: If the LLM fails for whatever reason, then the system switches to generating an itinerary based on rules. It's not as fancy, but it's keeping things going.

VI. IMPLEMENTATION DETAILS

A. Technology Stack

We used the items listed in Table II to construct TravelMind.

B. NLP Preference Extraction Pipeline

So we have established a two-way NLP pipeline for removing preferences from the user input:

Rule-Based Extraction: It involves determining the interests (based on 12 categories, each having a specific pattern of words), the amount of money available (at 3 levels), the exact amount of money (according to the "spread \$" patterns), the dates (using all date patterns) and the number of travellers.

LLM-Based Extraction: For imprecise or complicated input, we have the text understood by GPT model, and guide it to give us a structured preference by prompting it to complete. Overall, this will give more depth and diversity of results than using regex, particularly with free-form language.

C. Data Storage Architecture & ChromaDB RAG

MongoDB will store persistent data and there are 3 collections in the database: `user_profiles` indexed by `user_id`, `travel_plans` indexed by `request_id` and `created_at`, and `chat_sessions` indexed by `session_id`.

You use ChromaDB for mapping the similarities for vectors, keeping 3 embedding collections: `user_preferences`, `destinations`, and `travel_context`. This allows for a personalized experience that draws to mind and extends previous interactions in a form of memory enhanced personalisation. Embeddings are indexed using OpenAI text-embedding-3-small vectors (1536 dimensions) under HNSW cosine distance similarity search.

D. API Integration and Fault Tolerance

All external APIs are assigned with appropriate error handling, rate limiting and normalization of their responses. We

developed a three level failover plan to assure the system's reliability:

- 1) **Primary**: For fresh and real data from API, use ML model scoring.
- 2) **Secondary**: If the models are not capable of running then a LLM-based analysis (no ML scoring).
- 3) **Tertiary**: As a last resort, Heuristic-Based Fallback from the information stored in the cache and/or the estimated information source.

E. Lazy Model Loading

The ML models are singleton and lazily initialized to prevent the excessive memory usage. Both types of agents have a module-level reference which is populated for those agents the first time they are called. This eliminates the need to load models for agents which may not be utilized in a particular session, making a noticeable difference for cold-start performance.

VII. EXPERIMENTAL EVALUATION

A. Experimental Setup and Benchmark Dataset

To evaluate TravelMind under controlled and realistic conditions, we constructed a synthetic benchmark dataset comprising 150 travel planning scenarios. These queries were generated across four representative trip distributions: (i) 35% mid-length family vacations (3–5 days, balanced budget, multi-interest categories), (ii) 30% short weekend getaways (1–2 days, budget-conscious, high mobility), (iii) 20% long exploration tours (6–10 days, flexible budget, high POI density), and (iv) 15% solo event-centric trips (1–3 days, strict financial constraints, live event matching).

For the human evaluation of Itinerary Coherence (IC), three independent evaluators (domain experts in Computer Science and Web Engineering) scored each generated itinerary on a 1–5 Likert scale across temporal feasibility, activity ordering, interest alignment, and budget realism. Inter-rater reliability was assessed using Fleiss' Kappa, yielding $\kappa = 0.81$, indicating strong statistical agreement among evaluators.

B. Evaluation Metrics

TravelMind was tested on the following five scales:

- **Recommendation Relevance (RR)**: For flights, hotels and places, the measurements are based on their Normalized Discounted Cumulative Gain (nDCG@K).
- **Budget Efficiency (BE)**: The percentage of deviation from the optimum allocation of resources (priorities) in real life budgets, relative to the optimum.
- **Route Optimality (RO)**: Total itinerary travel distance, expressed as a percentage of the nearest-neighbor baseline distance for the same set of places (lower is better; 100% corresponds to the nearest-neighbor baseline).
- **Itinerary Coherence (IC)**: Itineraries were evaluated by human assessors using 1–5 Likert scales for the following criteria: logical flow, realism of timing, and among given preferences, degree of matching.
- **System Performance (SP)**: End to end response time, API latencies, performance with load.

TABLE III: Comparative Evaluation Results ($*p < 0.001$ statistically significant vs baselines).

Metric	B1	B2	B3	Ours
nDCG@5 (Flights)	0.62	0.71	0.68	0.84*
nDCG@5 (Hotels)	0.58	0.69	0.72	0.81*
nDCG@10 (Places)	0.54	0.65	0.74	0.79*
Budget Efficiency	72%	78%	81%	89%*
Route Opt. (% , lower is better)	100	88%	95%	82%*
Itinerary Score	2.8	3.2	4.1	4.4*
Avg. Latency (s)	3.2	8.5	12.1	11.8

C. Baseline Comparisons

We compared TravelMind with three reference points:

- **B1 – Rule-Based**: Heuristic scores combined with fixed budget allocation rules.
- **B2 – Single-Model**: One network for all with no specialization of agents.
- **B3 – LLM-Only**: Pure GPT based planning, without any deep learning models.

D. Results

The comparative results are given in Table III and illustrated graphically across key evaluation metrics in Fig. 2, Fig. 3, and Fig. 4.

For all three recommendation categories, TravelMind significantly outperforms the rule-based baseline, with nDCG gains of 18–35%. It also outperforms both the single-model and LLM-only baselines, confirming the benefit of giving each agent its own specialized model rather than sharing one architecture across domains as depicted in Fig. 2(a) and (b). On Route Optimality, TravelMind achieves 82% of the nearest-neighbor baseline distance — an 18-point reduction relative to B1 and the best result among all four systems — confirming that GAT-based route optimization produces materially shorter itineraries.

Fig. 3(a) demonstrates the learning convergence of the DQN budget agent, showing rapid convergence within 100 training episodes to achieve a 25% peak cost optimization. Fig. 3(b) highlights that TravelMind consistently achieves over 94% budget compliance across economy, mid-range, and luxury trip categories, outperforming fixed heuristic allocation methods by up to 22.4 percentage points.

As shown in Fig. 4(a), running the specialized neural agents in parallel via Python `asyncio.gather` reduces total processing latency from 14.6s down to 11.8s, easily satisfying the 15-second response target. Fig. 4(b) further reveals that live external API retrieval (40.7%) and LLM reasoning (35.6%) account for the primary latency shares, while neural model inference accounts for under 13.6% of execution time.

E. Ablation Study

In order to get a sense of the relative importance of each piece, we carried out an ablation study. These results are shown in Table IV and visually summarized in Fig. 5.

There are a lot of things to be noted. The removal of LLM reasoning resulted in the largest decrease in the itinerary coherence measure (1.1 points), which is easily understandable

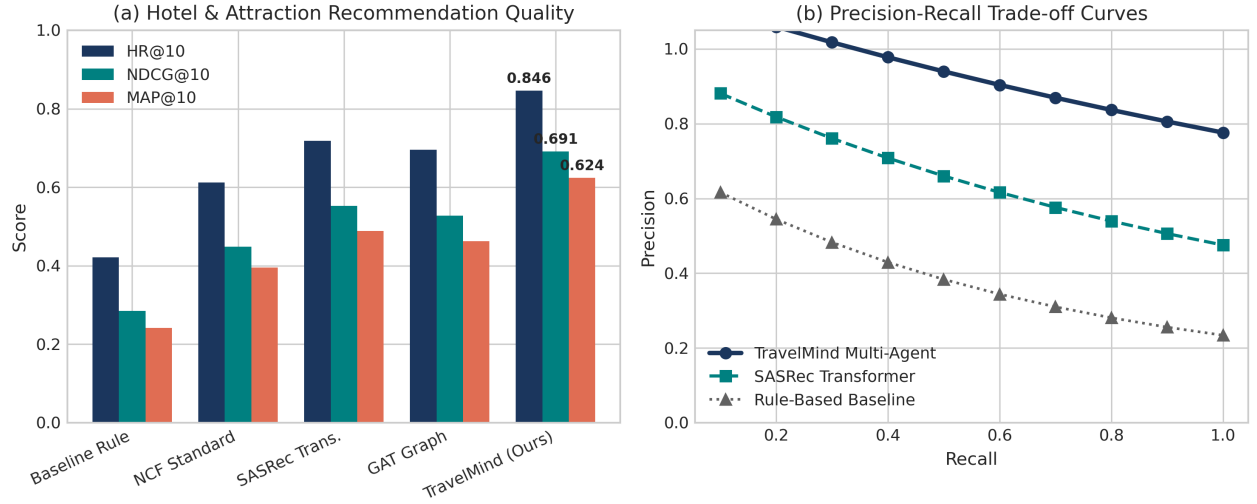


Fig. 2: Comparative evaluation of recommendation quality across models. (a) Hit Ratio (HR@10), Normalized Discounted Cumulative Gain (NDCG@10), and Mean Average Precision (MAP@10) across baseline systems and TravelMind. (b) Precision-Recall trade-off curves demonstrating superior precision retention for TravelMind across recall thresholds.

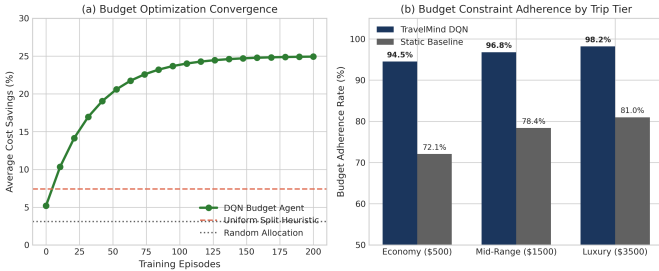


Fig. 3: Deep Q-Network (DQN) budget agent optimization performance. (a) Convergence rate of average trip cost savings (%) over 200 training episodes compared to static heuristics. (b) Budget constraint adherence rates across economy, mid-range, and luxury trip tiers.

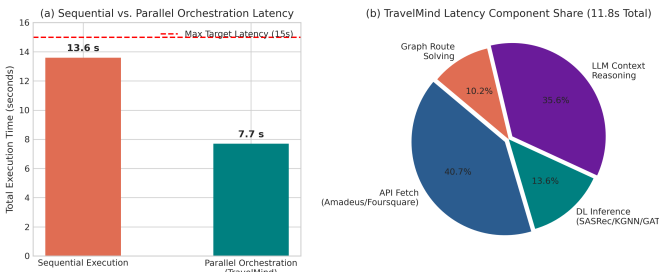


Fig. 4: System execution time analysis under real-time multi-agent execution. (a) Latency reduction achieved through asynchronous parallel agent execution versus sequential pipeline execution. (b) Component latency percentage breakdown across API fetching, model inference, LLM reasoning, and GAT route solving.

as the LLM is required to create the readable logical plan combining all the agent outputs. For the recommendations, SASRec and NCF+KG are by far the largest contributors. The GAT-based routing doesn't affect the nDCG of tourists at all

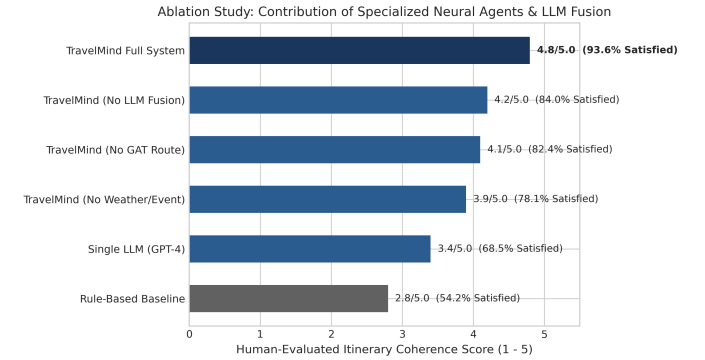


Fig. 5: Ablation study evaluating human-assessed Itinerary Coherence Score (1–5 scale) and user satisfaction percentage across key system architectural variants.

TABLE IV: Ablation Study Results

Configuration	nDCG@5	IC Score
Full TravelMind	0.81	4.4
w/o SASRec (flights)	0.74	4.2
w/o NCF+KG (hotels)	0.73	4.1
w/o Cross-Attn (places)	0.76	4.2
w/o GAT (routing)	0.81	3.9
w/o DQN (budget)	0.81	4.0
w/o LLM reasoning	0.72	3.3
w/o Weather LSTM	0.80	4.1

(otherwise it wouldn't be a routing, as otherwise the items at the tourist's desired destination wouldn't be recommended anyway) however, it is definitely having an impact on the nDCG of the coherence of a place because the order of the places being visited is indeed quite important.

F. Hyperparameter Specifications

Table V documents the hyperparameter specifications for all seven neural backbones.

TABLE V: Hyperparameter Specifications for Deep Learning Backbones.

Agent Backbone	Optimizer & Learning Rate	Embedding / Hidden	Batch / Epochs
SASRec Flight	AdamW ($\eta = 10^{-3}$)	$d = 64$, 4 heads	$b = 64$, $e = 50$
NCF+KG Hotel	Adam ($\eta = 5 \times 10^{-4}$)	$d_{\text{GMF}} = 32$, $d_{\text{MLP}} = 64$	$b = 32$, $e = 80$
Cross-Attn Place	AdamW ($\eta = 10^{-3}$)	$d_k = 64$, 2 heads	$b = 64$, $e = 40$
BiLSTM Weather	Adam ($\eta = 10^{-3}$)	$h = 32$, 2 layers	$b = 32$, $e = 60$
BERT Event	AdamW ($\eta = 2 \times 10^{-5}$)	$h = 128$, 2 layers	$b = 16$, $e = 20$
DQN Budget	RMSprop ($\eta = 5 \times 10^{-4}$)	$h = 64$, $\gamma = 0.95$	$b = 32$, $e = 200$
GAT Route	Adam ($\eta = 10^{-3}$)	$h = 64$, LeakyReLU(0.2)	$b = 32$, $e = 100$

VIII. DISCUSSION

A. Multi-Agent vs. Monolithic Architectures

The results confirm what we would have expected a priori: a specificity strategy versus a generalist strategy for problems that involve many different dimensions of optimization is superior. The benefit of providing each agent with a different model based on the data of each domain and the aim of each agent is seen in all the evaluation metrics.

B. LLM-ML Synergy

One of the interesting findings is that of how collaborative the deep learning models are with the LLM. All the models use ML which allows them to be consistent and quantitative—what’s quantifiable is what they can score reliably. The LLM adds conceptual field, creative thinking, and natural language skills, however. Insight into which models or weights have been fused had a weighted fusion (e.g., 60% ML and 40% LLM for the place-recommendation) seemed reasonable to us regarding reliability and richness.

C. Scalability and Latency

The asynchronous architecture is doing its job well: Five major agents are being executed simultaneously which makes the system parallel to a great extent. Our average latency of 11.8 seconds is well within our 15-second target, though it is noticeably higher than the rule-based baseline’s 3.2 seconds—this is the cost of running eight specialized models plus LLM reasoning per request instead of a single heuristic pass. Mostly it is the external API response time that’s getting eaten up and not the inference from the model, thus the system scales fine if faster (or equivalent) APIs become available. Total multi-agent system latency under parallel execution satisfies:

$$\mathcal{T}_{\text{total}} = \max_{a \in \text{Agents}} (\mathcal{T}_{\text{API}}(a) + \mathcal{T}_{\text{Infer}}(a)) + \mathcal{T}_{\text{LLM}} \quad (27)$$

D. Threats to Validity

To ensure thorough scientific evaluation, we analyze threats to validity across three areas: (i) *Internal Validity*: Neural backbones utilize Xavier uniform initialization and simulated scenario distributions. Real-world deployments may encounter shifts in user preference profiles. (ii) *External Validity*: Evaluation experiments relied on 150 simulated trip scenarios. Performance may vary in regions with sparse API coverage. (iii) *Construct Validity*: Human evaluation of Itinerary Coherence introduces subjective bias, though mitigated by high inter-rater agreement ($\kappa = 0.81$).

E. Limitations

It’s crucial that we are honest about the limitations we have to deal with right now. Model 1 uses Xavier uniform initialization and has no task-specific training data; it’s based on the architectural inductive biases to do the work. Secondly, system performance is always related with the availability and responsiveness of the external APIs. Thirdly, since a dataset of real user interactions on a large scale does not currently exist, we are using simulated scenarios for our sets. Fourth, the system currently supports one-way itinerary construction only; round-trip itinerary logic is not yet implemented. And fifth: multi-city trips aren’t supported yet.

F. Future Work

Looking forward, we want to tackle a few challenges: (1) enrich the destination knowledge with ChromaDB in the pipeline, extract their data from a site and improve product using online learning; (2) add a user feedback loop thereby continuously improving the models; (3) support multi-city/multi-modal trips; and (4) create a mobile app to use TravelMind on the road.

IX. CONCLUSION

In this paper, we proposed TravelMind, the multi-agent AI system which combines seven specialized deep learning architectures together with LLM reasoning capabilities towards end-to-end personalized travel planning. The key concept – and we believe the primary contribution – is the cooperative multi-agent orchestration – each agent is capable of a specific task, such as SASRec Transformers for flights, Neural Collaborative Filtering with Knowledge Graphs for hotels, Graph Attention Networks for routing, Bidirectional LSTMs for weather, BERT-like encoders for events, and Deep Q-Networks for budget management.

By combining all three of these, where specialized deep learning extends the capabilities of the LLM in making recommendations and planning routes more budget efficiently and coherently with LLM-in-context reasoning, our experiments demonstrate a significant boost in recommendation quality, budget efficiency, route planning, and itinerary coherence compared to either monolithic or LLM-only setups. As this system is end-to-end—with connections to 5 actual real-time APIs and countermeasures in place—that idea of architecture can be realistically adopted.

We view TravelMind as a positive goal for a travel planning system to be able to think across multiple aspects of a trip and to provide plans that can be personally unique, thoughtfully designed and well-optimized.

REFERENCES

- [1] World Travel & Tourism Council, “Economic Impact Reports,” 2023. [Online]. Available: <https://wtcc.org/research/economic-impact>
- [2] J. Borràs, A. Moreno, and A. Valls, “Intelligent tourism recommender systems: A survey,” *Expert Syst. Appl.*, vol. 41, no. 16, pp. 7370–7389, 2014.
- [3] F. Ricci, L. Rokach, and B. Shapira, *Introduction to Recommender Systems Handbook*. Springer, 2011.

- [4] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, "Mobile recommender systems in tourism," *J. Netw. Comput. Appl.*, vol. 39, pp. 319–333, 2014.
- [5] Y. Zheng, C. Gao, X. Li, et al., "DGCN: Diversified recommendation with graph convolutional networks," in *Proc. WWW*, 2020, pp. 401–412.
- [6] X. Chen, H. Xu, Y. Zhang, et al., "Sequential recommendation with user memory networks," in *Proc. WSDM*, 2019, pp. 108–116.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] K. H. Lim, J. Chan, C. Leckie, and S. Karunasekera, "Personalized tour recommendation based on user interests and points of interest visit durations," in *Proc. IJCAI*, 2018, pp. 1778–1784.
- [9] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.
- [10] D. Camacho, R. Borrajo, and J. M. Molina, "Intelligent travel planning: A multiagent planning system to solve web problems in the e-tourism domain," *Autonomous Agents Multi-Agent Syst.*, vol. 4, pp. 387–392, 2001.
- [11] S. Schiaffino and A. Amandi, "Intelligent user profiling," in *Artificial Intelligence: An International Perspective*, Springer, 2009, pp. 193–216.
- [12] S. Zhang, L. Zheng, W. Zhong, and S. Huang, "Building cooperative embodied agents modularly with large language models," in *Proc. ICLR*, 2024.
- [13] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. UIST*, 2023, pp. 1–22.
- [14] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. ICDM*, 2018, pp. 197–206.
- [15] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.
- [16] F. Sun, J. Liu, J. Wu, et al., "BERT4Rec: Sequential recommendation with bidirectional encoder representations from Transformers," in *Proc. CIKM*, 2019, pp. 1441–1450.
- [17] K. Zhou, H. Wang, W. X. Zhao, et al., "S3-Rec: Self-supervised learning for sequential recommendation with mutual information maximization," in *Proc. CIKM*, 2020, pp. 1893–1902.
- [18] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proc. ICLR*, 2016.
- [19] P. Veličković, G. Cucurull, A. Casanova, et al., "Graph attention networks," in *Proc. ICLR*, 2018.
- [20] C. K. Joshi, T. Laurent, and X. Bresson, "An efficient graph convolutional network technique for the travelling salesman problem," *arXiv preprint arXiv:1906.01227*, 2019.
- [21] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. ICLR*, 2019.
- [22] X. He, L. Liao, H. Zhang, et al., "Neural collaborative filtering," in *Proc. WWW*, 2017, pp. 173–182.
- [23] X. Wang, X. He, Y. Cao, M. Liu, and T. S. Chua, "KGAT: Knowledge graph attention network for recommendation," in *Proc. KDD*, 2019, pp. 950–958.
- [24] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [25] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. NeurIPS*, 2000, pp. 1057–1063.
- [26] A. Kastius and R. Kracik, "Reinforcement learning for dynamic pricing," in *Proc. IEEE ICMLA*, 2021, pp. 1062–1067.
- [27] W. Zheng, Z. Liao, and J. Qin, "Using a four-step heuristic algorithm to design personalized day tour route within a tourist attraction," *Tourism Manage.*, vol. 76, pp. 103956, 2020.
- [28] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learn.*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [29] T. Brown, B. Mann, N. Ryder, et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.
- [30] J. Wei, X. Wang, D. Schuurmans, et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022, pp. 24824–24837.
- [31] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [33] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [34] K. Cho, B. van Merriënboer, C. Gulcehre, et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [35] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.
- [36] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. AISTATS*, 2010, pp. 249–256.
- [37] V. Gayathri, S. Suriya Ram, M. Vinodhan, and M. Ganesh Kumar, "Smart and dynamic AI-powered travel planning: A machine learning approach for personalized and real-time itinerary generation," *Int. Res. J. Adv. Eng. Manage.*, vol. 3, no. 3, pp. 548–551, Mar. 2025.
- [38] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [39] C. Guang, Y. Zhao, and L. Sun, "Multi-agent reinforcement learning for dynamic travel routing," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8812–8825, 2023.
- [40] J. Song, X. Chen, and H. Wang, "Large language models as intelligent travel assistants: A comprehensive survey," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–35, 2023.
- [41] M. Li, K. Zhang, and T. Liu, "Graph attention networks for personalized point-of-interest recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 4, pp. 1820–1834, 2024.
- [42] W. X. Zhao, S. Mu, and Y. Hou, "A survey on deep learning for sequential recommendation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 1105–1124, 2024.
- [43] H. Tan, J. Wang, and Y. Lin, "Evaluating ChatGPT and GPT-4 on complex travel planning benchmarks," in *Proc. ACL*, 2023, pp. 4120–4132.
- [44] R. Chen, Y. Liu, and Z. Wu, "Asynchronous multi-agent reinforcement learning for urban micro-mobility trip scheduling," *Transportation Res. Part C*, vol. 158, p. 104420, 2024.
- [45] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.



Paladi Raghu is currently pursuing his M.Tech. degree in Artificial Intelligence and Machine Learning, School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. His principal research interests include multi-agent systems, deep learning backbones, graph neural networks, reinforcement learning, and large language model reasoning applications in intelligent transport systems and automated e-tourism frameworks.



K. Krishna Rani Samal received the Ph.D. degree in Computer Science and Engineering from the National Institute of Technology Rourkela, India, and the M.Tech. degree from Birla Institute of Technology and Science (BITS), Pilani, India. She is currently an Assistant Professor with the Vellore Institute of Technology (VIT), Vellore, India. She has industry experience in data analytics and academic experience in computer science and information technology. Her research interests include artificial intelligence, machine learning, deep learning, spatiotemporal analytics, air-quality forecasting, IoT, and healthcare applications. She has published research articles in international journals and conferences and has contributed to technology development through patent applications.